

(1) 跑通 2 倍速程序

首先改写了程序（见附件），使得其统计并输出了矩阵计算时间（这里将 M 设为 1024，便于观察时间，因为 4096 耗时太长了）

在 linux 虚拟机下运行原程序和 2 倍速加速程序，可以看到时间大致减少了一半

```
root@localhost:~  
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)  
[root@localhost ~]# gedit matrix.c  
[root@localhost ~]# gcc matrix.c -o matrix -O0  
[root@localhost ~]# ./matrix  
Initializing matrices...  
Starting matrix multiplication (M=1024)...  
  
Matrix multiplication complete.  
  
Time taken for calculation: 2.570000 seconds  
[root@localhost ~]# gedit matrix2.c  
[root@localhost ~]# gcc matrix2.c -o matrix2 -O0  
[root@localhost ~]# ./matrix2  
Matrices initialized. Starting fork and calculation (M=1024)...  
[Parent PID: 72630] Calculating rows 512 to 1023...  
[Child PID: 72631] Calculating rows 0 to 511...  
[Child PID: 72631] Finished calculation. matrixDes[0][0] = 1024  
[Parent PID: 72630] Child process terminated. All calculation done.  
  
Total calculation time: 1.290000 seconds
```

(2) 继续测试 4、8、16 倍速的程序（代码见附件）

采用宏定义形式，对于加速倍速只需修改 N 的值即可

4 倍速：

```
[root@localhost ~]# gedit matrix_N.c  
[root@localhost ~]# gcc matrix_N.c -o matrix4  
[root@localhost ~]# ./matrix4  
Matrices initialized. Starting 4 parallel calculations (M=1024)...  
[PID: 73062 | Rank: 1] Calculating rows 256 to 511...  
[PID: 73060 | Rank: 3] Calculating rows 768 to 1023...  
[PID: 73061 | Rank: 0] Calculating rows 0 to 255...  
[PID: 73063 | Rank: 2] Calculating rows 512 to 767...  
[PID: 73063 | Rank: 2] Finished calculation. matrixDes[512][0] = 1024  
[PID: 73062 | Rank: 1] Finished calculation. matrixDes[256][0] = 1024  
[PID: 73060 | Rank: 3] Finished calculation. matrixDes[768][0] = 1024  
[PID: 73061 | Rank: 0] Finished calculation. matrixDes[0][0] = 1024  
[Parent PID: 73060] All child processes terminated. All calculation done.  
  
Total calculation time with 4 processes: 0.690000 seconds
```

8 倍速：

```

[root@localhost ~]# gedit matrix_N.c
[root@localhost ~]# gcc matrix_N.c -o matrix8
[root@localhost ~]# ./matrix8
Matrices initialized. Starting 8 parallel calculations (M=1024)...
[PID: 73138 | Rank: 0] Calculating rows 0 to 127...
[PID: 73139 | Rank: 1] Calculating rows 128 to 255...
[PID: 73140 | Rank: 2] Calculating rows 256 to 383...
[PID: 73141 | Rank: 3] Calculating rows 384 to 511...
[PID: 73142 | Rank: 4] Calculating rows 512 to 639...
[PID: 73137 | Rank: 7] Calculating rows 896 to 1023...
[PID: 73143 | Rank: 5] Calculating rows 640 to 767...
[PID: 73144 | Rank: 6] Calculating rows 768 to 895...
[PID: 73143 | Rank: 5] Finished calculation. matrixDes[640][0] = 1024
[PID: 73137 | Rank: 7] Finished calculation. matrixDes[896][0] = 1024
[PID: 73144 | Rank: 6] Finished calculation. matrixDes[768][0] = 1024
[PID: 73141 | Rank: 3] Finished calculation. matrixDes[384][0] = 1024
[PID: 73142 | Rank: 4] Finished calculation. matrixDes[512][0] = 1024
[PID: 73138 | Rank: 0] Finished calculation. matrixDes[0][0] = 1024
[PID: 73139 | Rank: 1] Finished calculation. matrixDes[128][0] = 1024
[PID: 73140 | Rank: 2] Finished calculation. matrixDes[256][0] = 1024
[Parent PID: 73137] All child processes terminated. All calculation done.

Total calculation time with 8 processes: 0.370000 seconds

```

16 倍速:

```

[PID: 73190 | Rank: 4] Calculating rows 896 to 1023...
[PID: 73192 | Rank: 6] Calculating rows 384 to 447...
[PID: 73191 | Rank: 5] Calculating rows 320 to 383...
[PID: 73195 | Rank: 9] Calculating rows 576 to 639...
[PID: 73196 | Rank: 10] Finished calculation. matrixDes[640][0] = 1024
[PID: 73189 | Rank: 3] Finished calculation. matrixDes[192][0] = 1024
[PID: 73192 | Rank: 6] Finished calculation. matrixDes[384][0] = 1024
[PID: 73194 | Rank: 8] Finished calculation. matrixDes[512][0] = 1024
[PID: 73185 | Rank: 15] Finished calculation. matrixDes[960][0] = 1024
[PID: 73186 | Rank: 0] Finished calculation. matrixDes[0][0] = 1024
[PID: 73198 | Rank: 12] Finished calculation. matrixDes[768][0] = 1024
[PID: 73197 | Rank: 11] Finished calculation. matrixDes[704][0] = 1024
[PID: 73187 | Rank: 1] Finished calculation. matrixDes[64][0] = 1024
[PID: 73190 | Rank: 4] Finished calculation. matrixDes[256][0] = 1024
[PID: 73200 | Rank: 14] Finished calculation. matrixDes[896][0] = 1024
[PID: 73199 | Rank: 13] Finished calculation. matrixDes[832][0] = 1024
[PID: 73188 | Rank: 2] Finished calculation. matrixDes[128][0] = 1024
[PID: 73191 | Rank: 5] Finished calculation. matrixDes[320][0] = 1024
[PID: 73195 | Rank: 9] Finished calculation. matrixDes[576][0] = 1024
[PID: 73193 | Rank: 7] Finished calculation. matrixDes[448][0] = 1024
[Parent PID: 73185] All child processes terminated. All calculation done.

Total calculation time with 16 processes: 0.180000 seconds

```

可以看到程序的矩阵计算耗时基本以 2 倍减少。并且后续测试了 32 倍速、64 倍速、128 倍速，加速效果也很明显。但是当进程数更多，如 256、512 时，程序耗时反而增加了，比 64 倍速计算耗时更长！

这是因为 Linux 允许创建任意数量的进程（直到系统资源耗尽）。当进程数较少时，系统的瓶颈在于 CPU 等待主内存数据（而非纯粹的 CPU 计算）。通过 fork 创建更多进程，任务被分解得更细。当进程数进一步增加时，进程数过多，系统资源被开销耗尽，操作系统需要花费大量的 CPU 时间来创建、调度和管理（fork 和 wait）数百个进程，时间反而增加。

实验结果证明了并行计算中的一个核心思想：加速并非线性无限的。存在一个最佳并行度，超过这个点后，由于进程创建和上下文切换的开销超过了并行带来的收益，程序性能将急剧下降。

(3) 结合思考题 1 继续加速程序

在原始代码中，`produce` 函数中，循环变量是 `k`，这导致了对第二个矩阵 `matrixOriginal[k][column]` 的按列访问（`k` 在变，`column` 固定）。按列访问是内存不连续的，会触发大量的 L1/L2 缓存失效，迫使 CPU 频繁等待主内存 (RAM)，这是程序慢的主要原因。

所以，优化方法是采用 `i-k-j` 的循环顺序来最大化缓存局部性（代码见附件）

优化后的 4 倍速程序：

```
[root@localhost ~]# gcc matrix_N_optimized.c -o matrix4o
[root@localhost ~]# ./matrix4o
bash: ./matrix4o: 没有那个文件或目录
[root@localhost ~]# gcc matrix_N_optimized.c -o matrix4o
[root@localhost ~]# ./matrix4o
Matrices initialized. Starting 4 parallel calculations (M=1024)...
[PID: 75176 | Rank: 0] Calculating rows 0 to 255 (Cache Optimized)...
[PID: 75177 | Rank: 1] Calculating rows 256 to 511 (Cache Optimized)...
[PID: 75175 | Rank: 3] Calculating rows 768 to 1023 (Cache Optimized)...
[PID: 75178 | Rank: 2] Calculating rows 512 to 767 (Cache Optimized)...
[PID: 75176 | Rank: 0] Finished calculation. matrixDes[0][0] = 1024
[PID: 75177 | Rank: 1] Finished calculation. matrixDes[256][0] = 1024
[PID: 75175 | Rank: 3] Finished calculation. matrixDes[768][0] = 1024
[PID: 75178 | Rank: 2] Finished calculation. matrixDes[512][0] = 1024
[Parent PID: 75175] All child processes terminated. All calculation done.

Total calculation time (CACHE OPTIMIZED) with 4 processes: 0.380000 seconds
```

耗时从原来的 0.69s 加速到 0.38 秒，加速效果明显

附件 1: 原 matrix 程序

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h> // 用于计时

// 题目要求 M 的值为 4096
#define M 1024
// 全局变量: 用于存储矩阵
int version = 1;
int matrixOriginal[M][M];
int matrixDes[M][M];
// 矩阵乘法核心函数: 计算 matrixDes[row][column] 的值
void produce(int row, int column)
{
    // i 是局部变量, 作为循环计数器
    int i;
    // 矩阵自乘: matrixDes[row][column] = matrixOriginal[row] * matrixOriginal[column]
    // 目标矩阵 matrixDes 元素初始化为 0 的操作在 main 中完成
    for (i = 0; i < M; i++)
    {
        matrixDes[row][column] += matrixOriginal[row][i] * matrixOriginal[i][column];
    }
}

int main()
{
    // 局部变量: 用于循环计数
    int i, j;
    // 用于计时
    clock_t start_time, end_time;
    double cpu_time_used;
    // 1. 初始化矩阵
    printf("Initializing matrices...\n");
    for (i = 0; i < M; i++)
    {
        for (j = 0; j < M; j++)
        {
            // 设定原始矩阵的元素
            matrixOriginal[i][j] = 1;
            // 结果矩阵元素初始化为 0
            matrixDes[i][j] = 0;
        }
    }
    // 2. 开始计时
    printf("Starting matrix multiplication (M=%d)...\n", M);
```

```

    start_time = clock();
    // 3. 矩阵乘法计算
    // 外层循环：遍历结果矩阵 matrixDes 的每一行 (row)
    for (i = 0; i < M; i++)
    {
        // 内层循环：遍历结果矩阵 matrixDes 的每一列 (column)
        for (j = 0; j < M; j++)
        {
            produce(i, j);
        }
    }
    // 4. 结束计时
    end_time = clock();
    cpu_time_used = ((double)(end_time - start_time)) / CLOCKS_PER_SEC;
    // 5. 输出结果和时间
    printf("\nMatrix multiplication complete.\n");
    printf("\nTime taken for calculation: %f seconds\n", cpu_time_used);
    return 0;
}

```

附件 2: matrix_N

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>    // 用于计时
#include <unistd.h>   // 用于 fork() 和 exit()
#include <sys/wait.h> // 用于 wait()
#include <sys/types.h> // 用于 pid_t

#define M 1024
#define NUM_PROCESSES 1024
// 全局变量 (父子进程各有独立的副本)
int version = 1;
int matrixOriginal[M][M];
int matrixDes[M][M];
// 矩阵乘法核心函数
void produce(int row, int column)
{
    int i;
    for (i = 0; i < M; i++)
    {
        matrixDes[row][column] += matrixOriginal[row][i] * matrixOriginal[i][column];
    }
}
// 通用矩阵计算函数

```

```

void calculate_chunk(int rank, int num_procs)
{
    int i, j;
    // 每个进程计算的行数
    int rows_per_proc = M / num_procs;
    // 计算起始行和结束行 (不包含 to)
    int from = rank * rows_per_proc;
    int to = (rank == num_procs - 1) ? M : (rank + 1) * rows_per_proc;
    printf("[PID: %d | Rank: %d] Calculating rows %d to %d...\n", getpid(), rank, from, to - 1);
    for (i = from; i < to; i++)
    {
        for (j = 0; j < M; j++)
        {
            produce(i, j);
        }
    }
    // 输出该进程计算的第一个元素
    printf("[PID: %d | Rank: %d] Finished calculation. matrixDes[%d][0] = %d\n", getpid(), rank,
from, matrixDes[from][0]);
}

int main()
{
    int i, j;
    int parent_from_row;
    // 用于计时
    clock_t start_time, end_time;
    double cpu_time_used;
    // 1. 初始化矩阵 (在 fork() 前进行)
    for (i = 0; i < M; i++)
    {
        for (j = 0; j < M; j++)
        {
            matrixOriginal[i][j] = 1;
            matrixDes[i][j] = 0;
        }
    }
    printf("Matrices initialized. Starting %d parallel calculations (M=%d)...\n", NUM_PROCESSES,
M);
    // 多进程相关变量
    pid_t pids[NUM_PROCESSES - 1]; // 存储子进程 ID
    int rank;
    // 2. 开始计时 (在创建第一个子进程前开始)
    start_time = clock();
    // 3. 创建 NUM_PROCESSES - 1 个子进程

```

```

for (i = 0; i < NUM_PROCESSES - 1; i++)
{
    pids[i] = fork();
    if (pids[i] < 0)
    {
        perror("fork failed");
        return 1;
    }
    if (pids[i] == 0)
    {
        // 子进程逻辑: rank 从 0 开始
        rank = i;
        calculate_chunk(rank, NUM_PROCESSES);
        exit(0); // 子进程完成任务后必须退出
    }
    // 父进程会继续循环创建下一个子进程
}

// 4. 父进程逻辑
// 父进程计算最后一块数据, rank = NUM_PROCESSES - 1
rank = NUM_PROCESSES - 1;
parent_from_row = rank * (M / NUM_PROCESSES);
calculate_chunk(rank, NUM_PROCESSES);
// 5. 父进程等待所有子进程完成
for (i = 0; i < NUM_PROCESSES - 1; i++)
{
    waitpid(pids[i], NULL, 0);
}

printf("[Parent PID: %d] All child processes terminated. All calculation done.\n", getpid());
// 6. 结束计时 (父进程在所有子进程 wait() 之后)
end_time = clock();
cpu_time_used = ((double)(end_time - start_time)) / CLOCKS_PER_SEC;
// 7. 输出结果和时间
printf("\nTotal calculation time with %d processes: %f seconds\n", NUM_PROCESSES,
cpu_time_used);

// 输出父进程计算的第一行元素, 以确认计算正确性 (结果应为 M=1024)
printf("Verification check: matrixDes[%d][0] (Parent's part) = %d\n", parent_from_row,
matrixDes[parent_from_row][0]);

return 0;
}

```

附件 3: matrix_N_optimized

```

#include <stdio.h>
#include <stdlib.h>

```

```

#include <time.h>      // 用于计时
#include <unistd.h>     // 用于 fork() 和 exit()
#include <sys/wait.h>   // 用于 wait()
#include <sys/types.h> // 用于 pid_t

#define M 1024
#define NUM_PROCESSES 4
// -----
// 全局变量（父子进程各有独立的副本）
int version = 1;
int matrixOriginal[M][M];
int matrixDes[M][M];
// 通用矩阵计算函数（缓存优化版）
void calculate_chunk(int rank, int num_procs)
{
    // 循环变量提前声明，兼容 C89
    int i, j, k;
    // 每个进程计算的行数
    int rows_per_proc = M / num_procs;
    // 计算起始行和结束行（不包含 to）
    int from = rank * rows_per_proc;
    int to = (rank == num_procs - 1) ? M : (rank + 1) * rows_per_proc;
    printf("[PID: %d | Rank: %d] Calculating rows %d to %d (Cache Optimized)...\n", getpid(), rank,
from, to - 1);

    // *****
    // *** 核心优化部分：循环重排为 i-k-j 顺序 ***
    // *****

    // 1. i 循环（进程分工的行块）
    for (i = from; i < to; i++)
    {
        // 2. k 循环（乘法的中间项）
        for (k = 0; k < M; k++)
        {
            // 3. j 循环（最内层循环，确保按行访问）
            for (j = 0; j < M; j++)
            {
                // 核心计算：matrixDes[i][j] += matrixOriginal[i][k] * matrixOriginal[k][j]
                // 确保 matrixOriginal[k][j] 和 matrixDes[i][j] 都是连续访问，提高缓存命中率
                matrixDes[i][j] += matrixOriginal[i][k] * matrixOriginal[k][j];
            }
        }
    }

    // *****
    // 输出该进程计算的第一个元素

```

```

        printf("[PID: %d | Rank: %d] Finished calculation. matrixDes[%d][0] = %d\n", getpid(), rank,
from, matrixDes[from][0]);
    }
}

int main()
{
    // 循环变量提前声明
    int i, j;
    // 用于计时
    clock_t start_time, end_time;
    double cpu_time_used;
    // 父进程的起始行, 用于末尾的 printf 验证
    int parent_from_row;
    // 检查 M 是否能被进程数整除
    if (M % NUM_PROCESSES != 0)
    {
        fprintf(stderr, "Error: M (%d) must be divisible by NUM_PROCESSES (%d)\n", M,
NUM_PROCESSES);
        return 1;
    }
    // 1. 初始化矩阵 (在 fork() 前进行)
    for (i = 0; i < M; i++)
    {
        for (j = 0; j < M; j++)
        {
            matrixOriginal[i][j] = 1;
            matrixDes[i][j] = 0;
        }
    }
    printf("Matrices initialized. Starting %d parallel calculations (M=%d)...\n", NUM_PROCESSES,
M);
    // 多进程相关变量
    pid_t pids[NUM_PROCESSES - 1]; // 存储子进程 ID
    int rank;
    // 2. 开始计时 (在创建第一个子进程前开始)
    start_time = clock();
    // 3. 创建 NUM_PROCESSES - 1 个子进程
    for (i = 0; i < NUM_PROCESSES - 1; i++)
    {
        pid_t child_pid = fork();
        if (child_pid < 0)
        {
            perror("fork failed");
            return 1;
        }
    }

```

```

        if (child_pid == 0)
        {
            // 子进程逻辑: rank 从 0 开始
            rank = i;
            calculate_chunk(rank, NUM_PROCESSES);
            exit(0); // 子进程完成任务后必须退出
        }

        pids[i] = child_pid; // 父进程保存子进程 ID
        // 父进程会继续循环创建下一个子进程
    }

    // 4. 父进程逻辑
    // 父进程计算最后一块数据, rank = NUM_PROCESSES - 1
    rank = NUM_PROCESSES - 1;
    // 在 main 中记录父进程的起始行
    parent_from_row = rank * (M / NUM_PROCESSES);
    calculate_chunk(rank, NUM_PROCESSES);
    // 5. 父进程等待所有子进程完成
    for (i = 0; i < NUM_PROCESSES - 1; i++)
    {
        waitpid(pids[i], NULL, 0);
    }

    printf("[Parent PID: %d] All child processes terminated. All calculation done.\n", getpid());
    // 6. 结束计时 (父进程在所有子进程 wait() 之后)
    end_time = clock();
    cpu_time_used = ((double)(end_time - start_time)) / CLOCKS_PER_SEC;
    // 7. 输出结果和时间
    printf("\nTotal calculation time (CACHE OPTIMIZED) with %d processes: %f seconds\n",
NUM_PROCESSES, cpu_time_used);

    // 验证结果 (结果应为 M=1024)
    printf("Verification check: matrixDes[%d][0] (Parent's part) = %d\n", parent_from_row,
matrixDes[parent_from_row][0]);

    return 0;
}

```